



Digital Nurture 5.0

Name - Milind Verma

Reg No. - 23BCE11695

What is the Factory Method Design Pattern?

The Factory Method Pattern is a Creational Design Pattern used to create objects without directly using the `new` keyword in the client code. Instead, a factory class creates and returns the required object based on the user's request.

This pattern separates object creation from business logic, making the code more organized and easier to maintain.

Why Use the Factory Method Pattern?

- Reduces dependency between classes.
- Makes the code easy to maintain.
- Simplifies object creation.
- Makes it easy to add new object types in the future.

Applications

- Notification Systems
- Database Connections
- Logging Frameworks
- GUI Applications

Algorithm

1. Create a common interface named **Notification**.
2. Create classes that implement the interface.
3. Create a **NotificationFactory** class.
4. Accept the notification type.
5. Return the required notification object.
6. Call the notification method.
7. Display the output.

CODE :

```
Exercise 2: Implementing the Factory Method Pattern > Factory_Method_Pattern.java > Factory_Method_Pattern
1  interface Notification {
2      void notifyUser();
3  }
4
5  class EmailNotification implements Notification {
6
7      @Override
8      public void notifyUser() {
9          System.out.println(x: "Email Notification Sent Successfully.");
10     }
11 }
12
13 class SMSNotification implements Notification {
14
15     @Override
16     public void notifyUser() {
17         System.out.println(x: "SMS Notification Sent Successfully.");
18     }
19 }
20
21 class PushNotification implements Notification {
22
23     @Override
24     public void notifyUser() {
25         System.out.println(x: "Push Notification Sent Successfully.");
26     }
27 }
28
29 class NotificationFactory {
30
31     public Notification createNotification(String type) {
32
33         if (type == null) {
34             return null;
35         }
36
37         if (type.equalsIgnoreCase("EMAIL")) {
38             return new EmailNotification();
39         }
40
41         if (type.equalsIgnoreCase("SMS")) {
42             return new SMSNotification();
43         }
44
45         if (type.equalsIgnoreCase("PUSH")) {
46             return new PushNotification();
47         }
48
49         return null;
50     }
51 }
52
53 public class Factory_Method_Pattern {
54
55     public static void main(String[] args) {
56
57         NotificationFactory factory = new NotificationFactory();
58
59         Notification notification1 = factory.createNotification("EMAIL");
60         notification1.notifyUser();
61
62         Notification notification2 = factory.createNotification("SMS");
63         notification2.notifyUser();
64
65         Notification notification3 = factory.createNotification("PUSH");
66         notification3.notifyUser();
67     }
68 }
```

OUTPUT:

```
[Done] exited with code 0 in 0.491 seconds
```

```
[Running] cd "/Users/milindverma/Desktop/cognizant_assignment/Exercise 2: Implementing the Factory Method Pattern/" && j  
java && java Factory_Method_Pattern  
Email Notification Sent Successfully.  
SMS Notification Sent Successfully.  
Push Notification Sent Successfully.
```

```
[Done] exited with code 0 in 0.514 seconds
```